

CSE 306 - Computer Graphics

Project Report: A Path-Traced Ray Tracer in C++

Jai Ansh Singh Bindra

May 18, 2026



Figure 1: Final textured cat (3954 triangles) rendered with every implemented feature enabled: diffuse walls, indirect lighting, anti-aliasing, ray-triangle intersection accelerated by a BVH, Phong-interpolated vertex normals and UV texture mapping with inverse-gamma look-ups. 512×512, 1000 paths per pixel, 5 bounces. Rendering time: ≈ 43 s on the test machine. (Screenshoted)

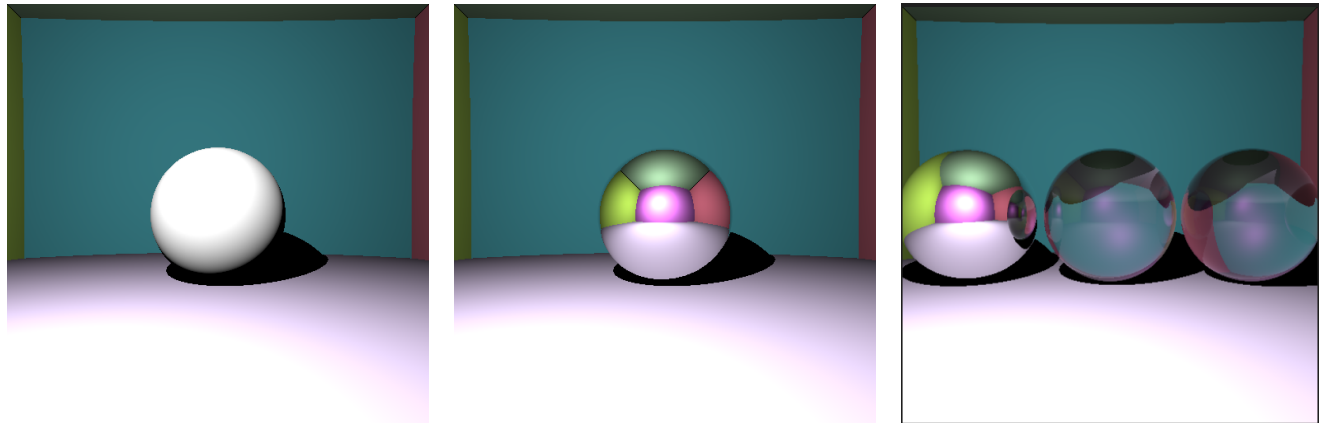
1 Introduction, Scope and Setup

In this project, I implement every mandatory feature of the assignment specifications: diffuse and mirror surfaces, direct lighting with hard shadows, indirect lighting without Russian roulette, anti-aliasing, ray-mesh intersection with Möller-Trumbore, and a Bounding Volume Hierarchy. In addition, I also implement the four flagged optional features: refractive (dielectric) surfaces with Fresnel-Schlick weighting, a thin-lens depth-of-field camera feature, Phong interpolation of vertex normals on triangle meshes (smoothing), and UV-mapped textures with gamma-correct sampling. All my main code is in the file (`main.cpp`) that builds with any C++17 or similar distribution (I by default use C++17). A second file, `main_optional.cpp`, contains the same code plus the texture mapping pipeline; I decided to fork this when the texture refactor temporarily broke the main build and I kept it as verbatim in repo root for transparency.

Hardware and parameters. All timings I report in this report were measured on my laptop equipped with a 12th-generation Intel Core i7-12650H (10 cores, 16 logical threads) and 16 GB of RAM under Windows 11. I compiled all my code with g++ (MSYS2, GCC 15.2) at `-O3 -fopenmp`; and the outer for over image rows is parallelised with `#pragma omp parallel for schedule(dynamic, 1)`. Unless otherwise stated, I tried to maintain uniformity and produced every image in this report at 512×512, a maximum recursion depth of 5, and the camera placed at (0, 0, 55). (Field of view is normally 60° and at most places I keep γ at 2.2). The light intensity is mostly $I = 6 \cdot 10^6$, except for the controlled comparison renders I do in Figures 3 and 4 where I raise it to $1 \cdot 10^7$ so that the “direct-only” panel is still legible. The sample-per-pixel count (`NB_PATHS`) varies and I specify it alongside each figure; and I provide an aggregate view in Table 2 at the end of the report.

2 Direct Illumination on Spheres

I started with a `Sphere` class, implementing the standard ray-sphere quadratic, taking the smallest root above $\varepsilon = 10^{-5}$ to avoid self-intersection on secondary rays. At every hit, I return P , N and t and apply the Lambertian shading $\rho \frac{I}{4\pi d^2} \frac{\max(0, N \cdot \omega_i)}{\pi} V(P)$ from lecture-notes, where $V(P) \in \{0, 1\}$ comes from a single shadow ray fired from $P + \varepsilon N$ towards the light (the offset is what kills shadow acne from what I understand). With just this implementation I already obtain a credible sphere inside the six-walled box, with the soft falloff towards the edges produced purely by inverse-square attenuation (Figure 2a).



(a) Diffuse sphere with point-light shading and a single shadow ray. 32 paths/pixel.

(b) Perfect mirror sphere: the recursive reflection picks up the coloured walls. 32 paths/pixel.

(c) Recreated reference scene: diffuse-mirror, solid glass and hollow glass sphere with Fresnel reflectance.

Figure 2: Three material types side by side. From left to right: pure Lambertian, perfect specular reflection, and dielectric transmission with Schlick's Fresnel weighting.

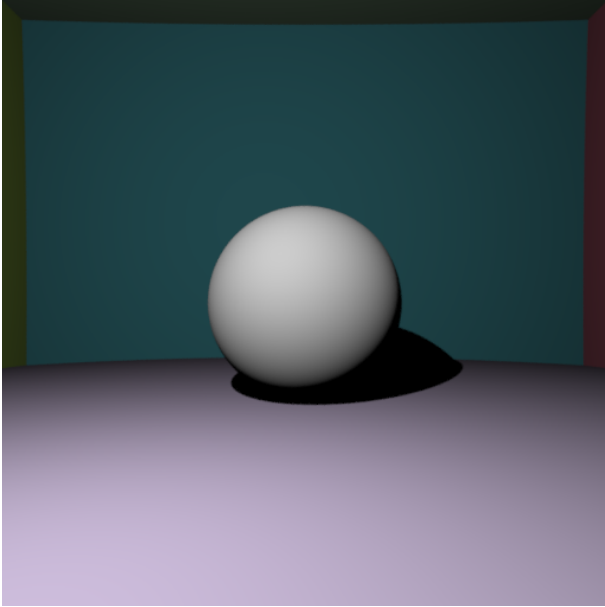
3 Reflective and Refractive Materials

Mirrors. A boolean flag on the hit object short-circuits shading into a recursive call: reflection direction $\omega_r = \omega - 2(\omega \cdot N)N$, new origin $P + \varepsilon N$, **depth+1**. The maximum-depth cap (5) guarantees termination. The coloured walls of the enclosing box being clearly visible inside the sphere (Figure 2b) was my sanity check that the reflection direction is indeed being computed in world space!

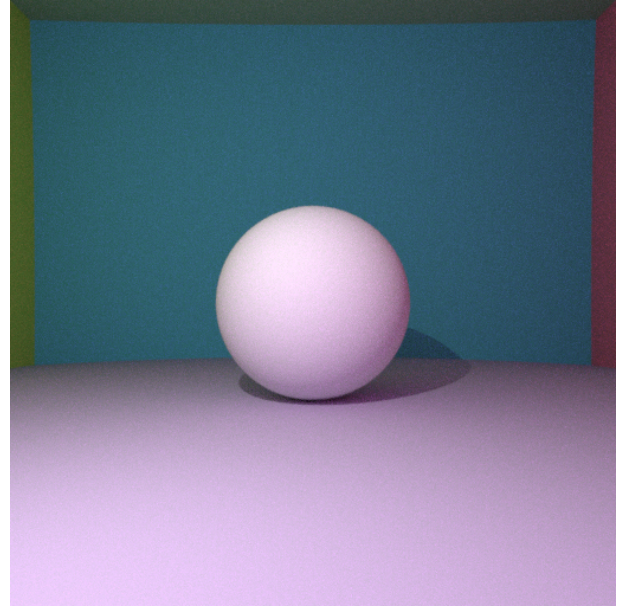
Transparent dielectrics (optional). I have implemented the Snell-Descartes, swap (n_1, n_2) and flip the working normal when the ray is exiting the medium, and detect total internal reflection as a negative discriminant (falling back to a pure mirror reflection). When refraction is valid, I weight reflection against transmission with Schlick's approximation and pick one branch by drawing a uniform random number against $F(\theta)$ - which is unbiased and avoids doubling the recursion fan-out at every interface. In Figure 2c, I almost (except the 3rd sphere) reproduce the reference scene with a solid glass sphere ($n = 1.5$) and a hollow glass sphere (two nested spheres of slightly different radius with inverted normals on the inner one: I ran out of time to fully correct this one compared to lecture-notes) given in lecture-notes.

4 Global Illumination by Monte Carlo Integration

I add the indirect term by sampling one cosine-weighted secondary direction at every diffuse hit, using the local tangent frame construction from the notes. Because the sampling density is proportional to $\cos \theta$, the $\cos \theta$ and π factors in the diffuse BRDF cancel and the indirect contribution reduces to a recursive multiplication of the albedo with the colour returned by the secondary ray; I mostly cap recursion at 5 bounces. Figure 3 shows what this single addition does on a Lambertian sphere, with the rest of the scene and the RNG seeds held fixed: the underside of the sphere lifts out of pitch black and the green/red/violet hues of the walls visibly bleed/show onto the floor and the lower half of the sphere.



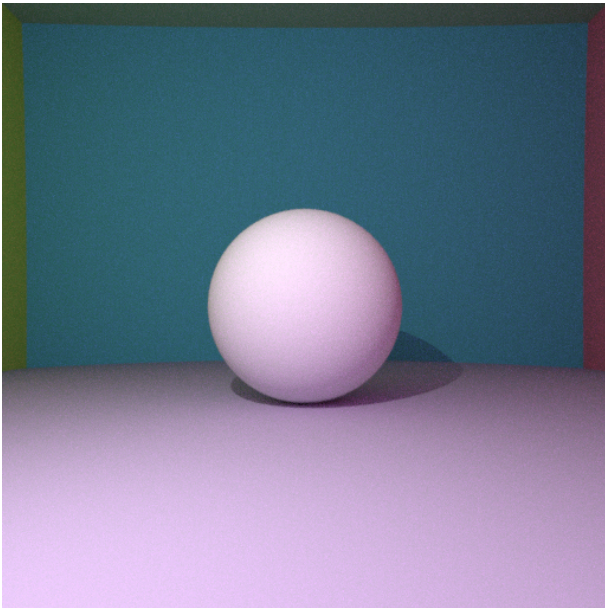
(a) **Indirect OFF.** Sphere bottom and box corners are pitch black; walls show only their direct diffuse term.



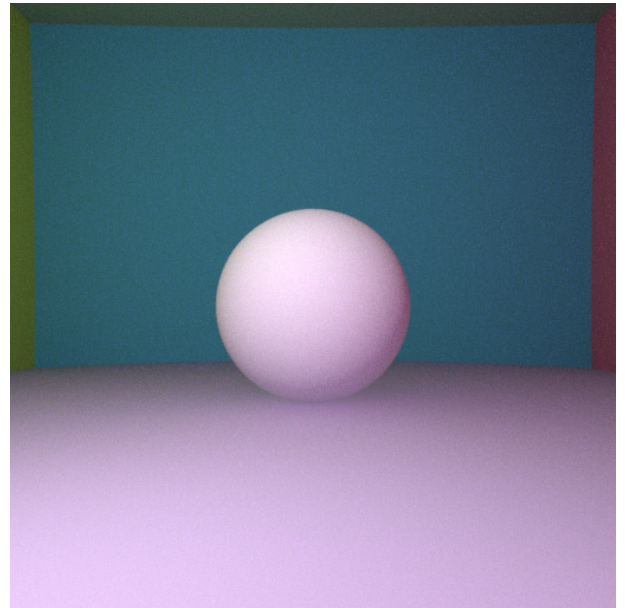
(b) **Indirect ON.** One extra cosine-weighted bounce per hit lifts the floor and bleeds wall hues. 0.37s vs. 2.48s.

Figure 3: Same scene at 64 paths/pixel with indirect lighting toggled. Both panels from `render_extras.cpp`, identical seeds.

Shadows. Toggling/Commenting out the visibility check (Figure 4) isolates the single line of code that produces all shadowing: the sphere itself is unchanged because $\max(0, N \cdot L)$ already attenuates its back, but the dark blob on the floor - the only $V(P) = 0$ region - disappears entirely.



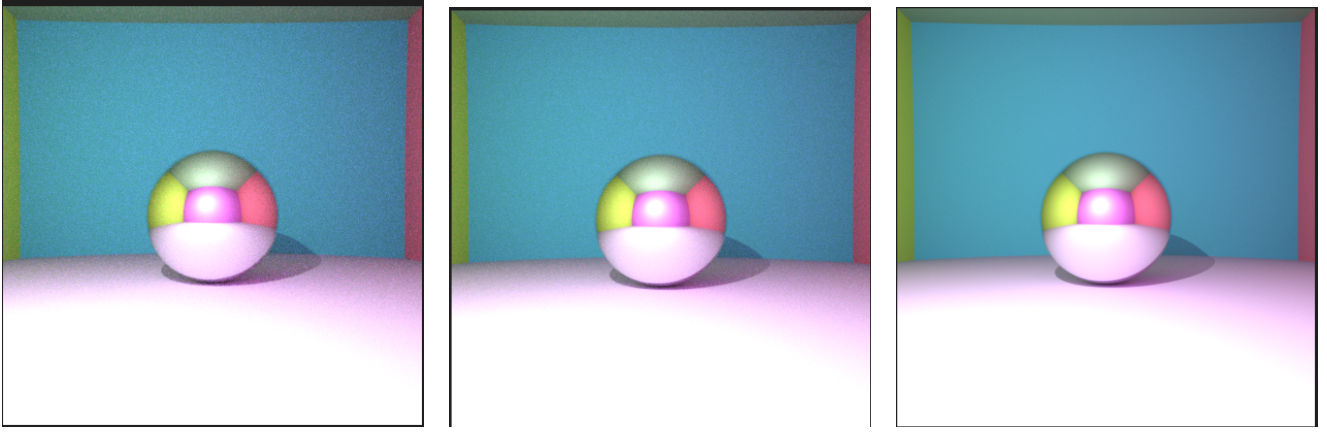
(a) **Shadow ray ON.** $V(P) \in \{0, 1\}$ blackens floor pixels whose line to the light is occluded.



(b) **Shadow ray OFF.** Shadow blob disappears; the rest is unchanged because $\max(0, N \cdot L)$ already attenuates the back.

Figure 4: Same scene as Figure 3, both with indirect lighting on; only the shadow ray toggles.

Anti-aliasing. Now, I draw a 2D Gaussian sub-pixel jitter ($\sigma = 0.5$ px) with Box-Muller inside the same sampling loop and add it to the pixel coordinates before constructing the primary ray. Because the same sample budget integrates over the pixel area, the BRDF, and (in Section 5) the lens disc all at once, variance falls as $1/N$ in `NB_PATHS`. I clearly demonstrate the convergence through Figure 5: at 32 paths the mirror sphere has a grainy halo and the shadow is speckled; at 64 it seems markedly cleaner; at 1000 both defects are (mostly) gone, at roughly $30\times$ the compute time.



(a) 32 paths/pixel: visible grain, jaggy silhouette. (b) 64 paths/pixel: $\sim \sqrt{2} \times$ less noise, close to converged on flat regions. (c) 1000 paths/pixel: noise below printing threshold.

Figure 5: Mirror-sphere scene at three sample counts. Variance falls as $1/N$; the right-most takes $\approx 30 \times$ the compute of the left.

Reproducibility under OpenMP. (Briefly what I implemented) Each thread gets its own `std::default_random_engine` from a 32-slot per-thread array; `omp_get_thread_num()` is threaded through `getColor` & `random_cos` so there are no races on the random state & each run is exactly reproducible.

5 Thin-Lens Depth of Field (Optional)

Here, I replaced the pinhole camera with a thin-lens model: each primary ray first hits the focal plane at $z = -55$, then I jitter its origin uniformly on a disc of radius $r_{\text{aperture}} = 3$ around the camera & recompute the direction from that jittered origin to the unchanged focal point. Spheres on the focal plane stay sharp; foreground and background spheres pick up bokeh-like blur (Figure 6).

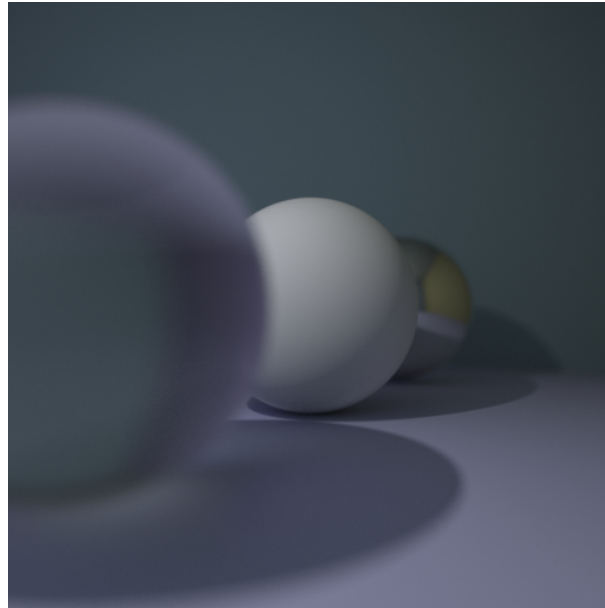


Figure 6: Three spheres along the optical axis with DoF enabled ($r_{\text{aperture}} = 3$, focal distance = 55). The Lambertian sphere on the focal plane is crisp; foreground transparent and background yellow spheres blur out. 1000 paths/pixel.

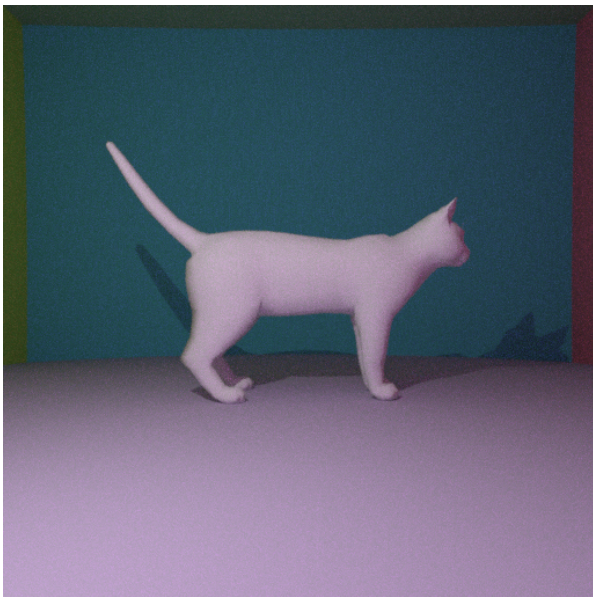
6 Polygonal Meshes via Möller-Trumbore

I completed the `TriangleMesh` class behind the same `Object` interface as `Sphere`. My `readOBJ` parses `v`, `vn`, `vt` and all four common face formats (`v`, `v/vt`, `v//vn`, `v/vt/vn`), resolves 1-based and negative OBJ indices, and fan-triangulates polygon faces. The supplied `cat.obj` (by Prof. Bonneel) parses into **3954** triangles. Ray-triangle intersection uses Möller-Trumbore: I recover barycentric (α, β, γ) by Cramer's rule and accept hits with all three in $[0, 1]$, $t > \epsilon$. As a first acceleration I gate the triangle loop behind a

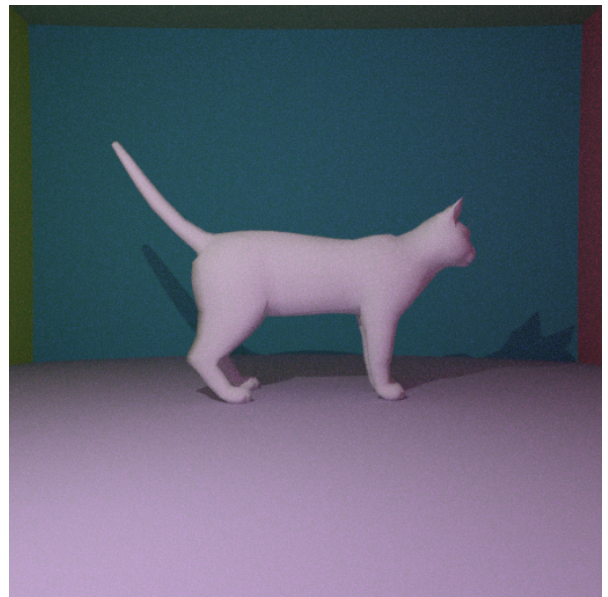
single axis-aligned bounding box intersected with the slab method on each axis which is already enough to keep most pixels of the framebuffer out of the triangle loop entirely. This is the baseline against which I measure the BVH (Bounding Volume Hierarchy) speed-up below.

7 Acceleration: Bounding Volume Hierarchy

A single bounding box still leaves every ray that hits it doing $O(T)$ work. To get the expected $O(\log T)$, I built a top-down BVH: `build_bvh` takes a half-open triangle range, computes its bounding box, partitions in place along the longest axis of the diagonal by triangle barycenter, and recurses on the two halves. I stop at leaves holding fewer than 5 triangles (so traversal cost is amortised) and on degenerate partitions. The whole tree for the cat builds in ≈ 1.4 ms (from my last local measurement). At render time I traverse iteratively with an `std::list<BVHNode*>` stack and push a child only if (a) its bounding box is hit and (b) the entry t is below the best triangle distance found so far: (b) is what does most of my pruning, since once the cat is hit any farther sub-tree is irrelevant. Leaves run Möller-Trumbore only on their own triangle range. The result is Table 1; Figure 7 confirms visually that the BVH does not change the picture, only the time it takes to produce it (which is reduced significantly).



(a) **Single bounding box only.** Every ray entering the cat's box tests all 3954 triangles. 76.25 s.



(b) **Full BVH (leaf size < 5).** Same scene, same seeds, same paths/pixel. 2.71 s.

Figure 7: BVH does not change the picture, only the time to produce it. Both panels are byte-equivalent up to parallel-reduction reordering. 512×512, 32 paths/pixel, 5 bounces.

Acceleration	Triangles	Render time	Speed-up	Report panel
Single bounding box only	3954	76.25 s	1.00×	Figure 7a
Full BVH (leaf size < 5)	3954	2.71 s	28.1×	Figure 7b

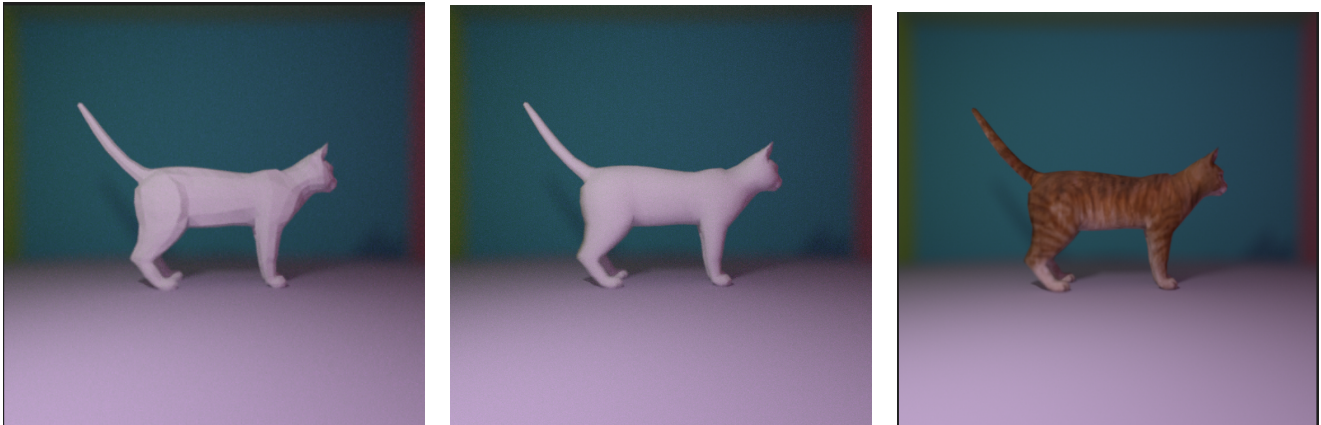
Table 1: Wall-clock time, same 512×512 image at `NB_PATHS = 32`, 5 bounces, 16-thread OpenMP, i7-12650H. The benchmark programs emitted raw PNGs, copied into the report as the two panels of Figure 7; DoF is off so only mesh traversal differs. BVH build is ≈ 1.4 ms, negligible.

The 28× speed-up is consistent with the assignment's/notes' reference numbers (3 s with BVH vs. ~ 9 minutes without on the reference machine); the absolute times here are smaller mainly because of the 16-thread CPU. The BVH is what makes any of the heavier renders in this report - 1000 paths/pixel, textures, indirect bounces - tractable in seconds rather than minutes.

8 Smooth Shading and Textures (Optional)

I reuse the barycentric coordinates from Möller-Trumbore twice without recomputation. First, if vertex normals are present, I set the shading normal to $N(P) = \alpha N_A + \beta N_B + \gamma N_C$ and renormalise, the silhouette is unchanged but the diffuse shading looks smooth across the body. Second, in `main_final_with_optional.cpp`, the same weights interpolate the per-vertex UVs and I look up a pixel

in `cat_diff.png` (loaded once at startup with `stbi_load`). UVs are wrapped to $[0, 1]$ so out-of-range coordinates tile rather than crash, and I apply $c \mapsto (c/255)^{2.2}$ on read so the sRGB-stored albedo is in linear space before the BRDF multiplies it.



(a) **Flat normals.** Per-face normals from $e_1 \times e_2$: the polygonal facets are clearly visible across the body.

(b) **Phong-interpolated normals.** Same mesh, same camera; the silhouette is unchanged but the shading looks smooth.

(c) **Phong + UV texture.** Barycentric weights also sample `cat_diff.png` with inverse gamma applied on read.

Figure 8: Appearance pipeline for the cat. (a) geometric normal only; (b) Phong-interpolated vertex normals; (c) Phong + gamma-correct UV texture lookup. All three reuse the same barycentric weights from Möller-Trumbore.

A short debugging note. On the first try I applied the lookup *without* inverse gamma, hence every indirect bounce then squared the already-bright red channels and saturated the cat into a uniform hot pinkish colour. Adding the $(c/255)^{2.2}$ correction on read collapses the issue. I reproduced it in pink color for fun later! (Figure 9)

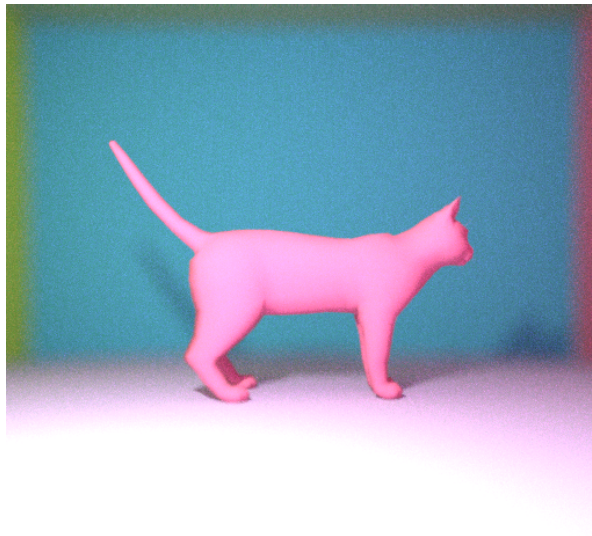


Figure 9: Recreated a pink coloured cat for fun later!.

9 Performance Summary

Through Table 2, I collect every render in the report on a single page. Rows tagged **measured** are direct `std::chrono::high_resolution_clock` timings emitted by local scripts I wrote for getting official rendering times; rows tagged “estimated” were either timed using a stopwatch by me (majorly) or my attempt to extrapolate them linearly from a measured baseline at the same resolution and bounce count (only a few).

Figure	Scene	Paths/pixel	Bounces	Time	Notes
1	Textured cat, BVH, full pipeline	1000	5	≈ 43 s	measured
2a	Diffuse sphere only	32	5	≈ 1 s	estimated
2b	Mirror sphere	32	5	≈ 1 s	estimated
2c	3-sphere reference scene	64	5	≈ 3 s	estimated
3a	Direct only (no indirect)	64	5	0.37 s	measured
3b	Direct + indirect + shadow	64	5	2.48 s	measured
4b	Direct + indirect, no shadow	64	5	2.28 s	measured
5a	Mirror sphere, 32 paths	32	5	≈ 1.4 s	estimated
5b	Mirror sphere, 64 paths	64	5	≈ 2.8 s	estimated
5c	Mirror sphere, 1000 paths	1000	5	≈ 43 s	estimated
6	DoF, 3 spheres along optical axis	1000	5	≈ 40 s	estimated
7a	Cat, single bbox only	32	5	76.25 s	measured
7b	Cat, full BVH	32	5	2.71 s	measured
8a, 8b	Cat, flat / smooth normals	64	5	≈ 3 s each	estimated
8c	Cat, smooth + texture (BVH)	1000	5	≈ 43 s	estimated

Table 2: Wall-clock summary of every render. Same 16-thread OpenMP build on the i7-12650H from Section 1. BVH-speedup and controlled-comparison rows are direct `std::chrono` measurements.

Two take-aways: the BVH dominates ($28\times$ at only $\sim 4k$ triangles) as expected, and within a fixed acceleration structure render time scales almost exactly linearly in paths-per-pixel and bounce depth, which the 32/64/1000-sample triplet in Figure 5 also shows.

10 Conclusion

My ray tracer covers every mandatory feature and *all four* of the suggested optional ones (Fresnel-Schlick refraction, thin-lens depth of field, Phong normal interpolation, gamma-correct UV textures), all in 2 .cpp files, mostly parallelised with OpenMP. BVH (which turns a 76-second cat render into a 2.7-second one) and the cosine-weighted hemisphere sampler (which makes the indirect bounces affordable) were 2 great design choices provided during this assignment! The cleanest reuse I make, I think, is the barycentric coordinates: the same (α, β, γ) computed once in Möller-Trumbore drives the intersection test, the smooth shading normal, and the texture lookup.

11 References and Footnote

Please note that some images in this report might have visible black edges/borders. They don't arise from a malfunction of my ray-tracer, they simply exist due to the fact that I took screenshots of a lot of my images once I generated them and I stored them in a folder ('images_archive/') at the root of my repo and it was not possible to perfectly select the area in my code editor!

I assure that each and every line of code in this project was either a part of the template code provided to us during the lab sessions or was written by me without the use of external agents. Whenever I ran into troubles, my main source of getting any debugging help/isnight was Stack Overflow (which has been my long-term guru since I have been passionate about programming since a long time now!) since it helps to get insights on how programmers solve similar problems as the ones we are stuck in.

All my code and images are available in my repository -
github.com/JaiAnshSB26/raytracer_computer_graphics

Lastly I would like to thank Prof. Bonneel and Mr. Gong for the opportunity to work on this project, I sincerely enjoyed doing it, and I hope you would like my work!